Object Oriented Programming

-- Course Review

(MUST-CS111)

Prof. Jinyu Tian

jytian@must.edu.mo

https://www.must.edu.mo/scse/staff/tianjinyu

About the Course

Assessment Method	Weight %
1.Attendance (Class participation)	8%
2.Assignments	50%
3. Quizzes	12%
4.Final exam	30%
Total	100 %

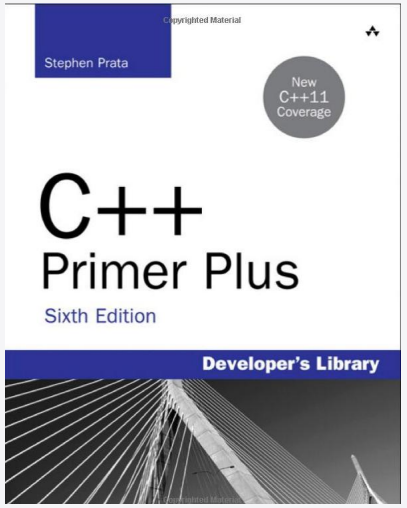
Course WeChat Group

Useful websites:

- https://en.cppreference.com/w/
- https://www.w3schools.com/cpp/
- http://cpp.sh/
- https://www.onlinegdb.com/

Suggested Textbook:

Stephen Prata. C++ Primer Plus, 5th or 6th Edition, Pearson, 2012. ISBN：978-0321-77640-2



Reference Books:

1. Bruce Eckel. Thinking in C++, Volume 1: Introduction to Standard C++, 2nd Edition, Prentice Hall, 2000, ISBN：9780139798092
2. Bruce Eckel. Thinking in C++, Volume 2: Practical Programming, 1st Edition, Prentice Hall, 2008, ISBN：9780131225527

1

Datatype and Operators

1.1 Integer Numbers

Integer Numbers

- **int** is the most frequently used integer type

```
int i; //declare a variable
int j = 10; //declare and initialize
int k;
k = 20; //assign a value
```

- Remember to initialize a variable!
- Will the compiler give an error?

```
int i;
cout << i; //what is i's value?
```

How to initialize

```
int num;
num = 10;//do not forget this line
int num = 10;
int num(10);
int num{10};
int num{ }; //Zero-initialization
```

char

- `char`: type for character, 8-bit integer indeed!
- `signed char`: signed 8-bit integer
- `unsinged char`: unsigned 8-bit integer
- `char`: either `signed char` or `unsinged char`

Some library Functions for “char”

The header file “<cctype>” defines several functions to manipulate characters. This header file belongs to the Standard Library Header of C++. Thus you should use them under the namespace std;

Function	Description
int isalnum(int ch);	checks if a character is alphanumeric
int isalpha (int ch);	checks if a character is alphabetic
int islower (int ch);	checks if a character is lowercase
int isupper (int ch);	checks if a character is an uppercase character

<cctype> in fact is the header file <ctype.h>.we used in C language.
More functions can refer to https://en.cppreference.com/w/cpp/header/cctype

Example

```
#include<iostream>
#include<cctype>
using namespace std;
int main(){
    char x;
    while(cin>>x){
    if(isalpha(x)){
        cout<<"The imput character is an alphabetic"<<endl;
    }else{
        cout<<"The imput character is not an alphabetic"<<endl;
    }
    }
}
```

```
@
The imput character is not an alphabetic or a number
```

bool

- *A C++ keyword, but not a C keyword*
- bool width: 1 byte (8 bits), NOT 1 bit!
- Value: *true* (1) or *false* (0)

1.2 Floating-point Numbers

What's the output?

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    float f1 = 1.2f;
    float f2 = f1 * 10000000000000000; //1.0e15
    cout << std::fixed << std::setprecision(15) << f1 << endl;
    cout << std::fixed << std::setprecision(1) << f2 << endl;
    return 0;
}
```

Understanding Computing

- Are computers always accurate?
- Floating-point operations always bring some tiny errors.
- Those errors cannot be eliminated.
- What we can do: to manage them not to cause a problem.

1.3 Arithmetic Operators

Arithmetic Operators

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo Operation (Remainder after division)

Assignment Operators

Operator	Example	Equivalent to
`=`	`a = b;`	`a = b;`
`+=`	`a += b;`	`a = a + b;`
`-=`	`a -= b;`	`a = a - b;`
`*=`	`a *= b;`	`a = a * b;`
`/=`	`a /= b;`	`a = a / b;`
`%=`	`a %= b;`	`a = a % b;`

Relational Operators

Operator	Meaning	Example
`==`	Is Equal To	`3 == 5` gives us **false**
`!=`	Not Equal To	`3 != 5` gives us **true**
`>`	Greater Than	`3 > 5` gives us **false**
`<`	Less Than	`3 < 5` gives us **true**
`>=`	Greater Than or Equal To	`3 >= 5` give us **false**
`<=`	Less Than or Equal To	`3 <= 5` gives us **true**

Logical Operators

Operator	Example	Meaning
&&	expression1 **&&** expression2	Logical AND. True only if all the operands are true.
||	expression1 **||** expression2	Logical OR. True if at least one of the operands is true.
!	**!**expression	Logical NOT. True only if the operand is false.

Note! We can equivalently use operators *and*, *or* and *not* as replacement.

Other Operators

Operator	Description	Example
`sizeof`	returns the size of data type	`sizeof(int); // 4`
`?:`	returns value based on the condition	`string result = (5 > 0) ? "even" : "odd"; // "even"`
`&`	represents memory address of the operand	`# // address of num`
`.`	accesses members of struct variables or class objects	`s1.marks = 92;`
`->`	used with pointers to access the class or struct variables	`ptr->marks = 92;`
`<<`	prints the output value	`cout << 5;`
`>>`	gets the input value	`cin >> num;`

"::" – scope operator

e.g.,

- std::cout
- Class::MemberFunction();

scope operator has the highest precedence

1.4 Type Inference

Keyword: auto

auto is placeholder type specifier.

The type of the variable will be deduced from its initializer.

```
auto a = 2; // type of a is int
auto b = 2.3; // type of b is double
auto c ; //valid in C, error in C++
auto d = a * 1.2;
```

- Question:

```
auto a = 2; // type of a is int

// will a be converted to a
//   double type variable?
a = 2.3;
```

No! 2.3 will be converted to a `int` 2, then assigned to `a`

Operator: decltype

Inspects the declared type of an entity or the type and value category of an expression.

Syntax

decltype (entity) (1) (since C++11)

decltype (expression) (2) (since C++11)

Operator: decltype

```cpp
#include <iostream>
#include <type_traits>
#include <string>
using namespace std;
class MyClass
{
private:
    int a;
    float b;
public:
    int get_a(){return a;}
};

struct MyStruct
{
    float y;
    string s;
};
```

```cpp
int main()
{
    int i = 33;
    MyClass my_class_1;
    MyStruct my_struct_1;
    decltype(i) j = i * 2;
    decltype(my_class_1) my_class_2(my_class_1);
    decltype(my_class_1.get_a()) x = 4;
    decltype(my_struct_1.s) my_string = "aaaa";
}
```

```
VARIABLES
  Locals
    i: 33
  > my_class_1: {...}
  > my_struct_1: {...}
    j: 66
  > my_class_2: {...}
    x: 4
    my_string: "aaaa"
```

Variables under Debug

2

COMPOUND TYPES

2.1 Arrays

Arrays

- A contiguously allocated memory
- Fixed number of objects (The array size cannot be changed)
- Its element type can be any fundamental type (int, float, bool, etc), structure, class, pointer, enumeration,

```
int num_array1[5]; //uninitialized array, random values
int num_array2[5] = {0, 1, 2, 3, 4}; //initialization
int num_array3[5] = {}; //zero initialization
```

2.2 String (C-string)

Array-style strings

- An array-style string (null-terminated strings/arrays of characters) is a series of characters stored in bytes in memory.
- This kind of strings can be declared as follows

```
char rabbit[16] = {'P', 'e', 't', 'e', 'r'};
char bad_pig[9] = {'P', 'e', 'p', 'p', 'a', ' ', 'P', 'i', 'g'}; //a
bad one!
char good_pig[10] = {'P', 'e', 'p', 'p', 'a', ' ', 'P', 'i', 'g', '\0'};
```

Some library Functions for "c-string"

The header file "<cstring>" defines several functions to manipulate C strings and arrays. This header file belongs to the Standard Library Header of C++. Thus you should use them under the namespace std;

- Copy

```
char* strcpy( char* dest, const char* src );
```

- Concatenate: appends a copy of src to dest

```
char *strcat( char *dest, const char *src );
```

- Compare

```
int strcmp( const char *lhs, const char *rhs );
```

- Get length

```
int strlen( const char *s);
```

<cstring> in fact is the header file <string.h> we used in C language.
More functions can refer to https://cplusplus.com/reference/cstring/

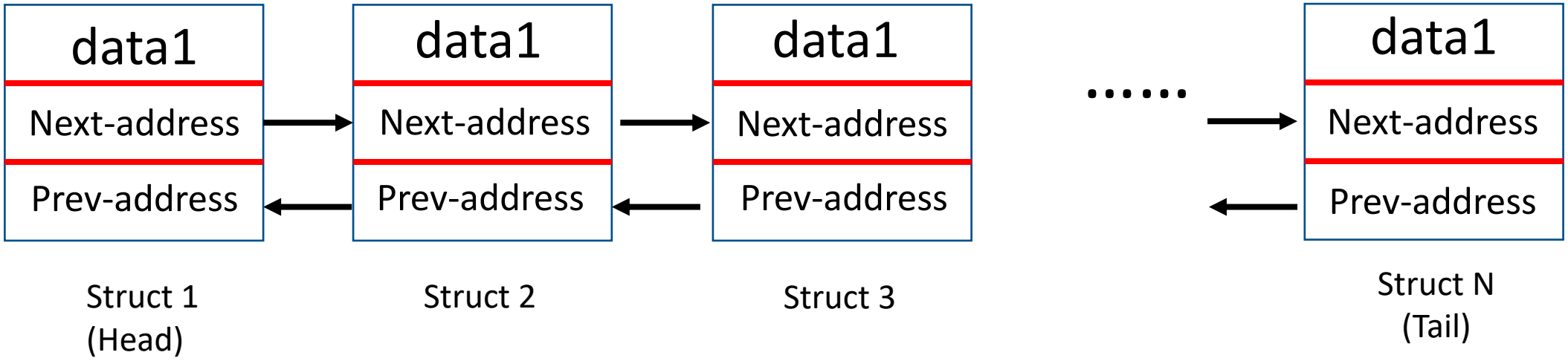
Example of Functions in <cstring>

```
#include <cstring>
#include<iostream>
int main()
{
    char rabbit[16] = {'P', 'e', 't', 'e', 'r'};
    char pig[20] = "peggy";
    std::cout<<"The length of rabbit is: "<<std::strlen(rabbit)<<std::endl;
    std::strcpy(rabbit,pig);
    std::cout<<"Copy the rabbit to the pig: "<<rabbit<<std::endl;
}
```

2.3 Linked List

Linked List

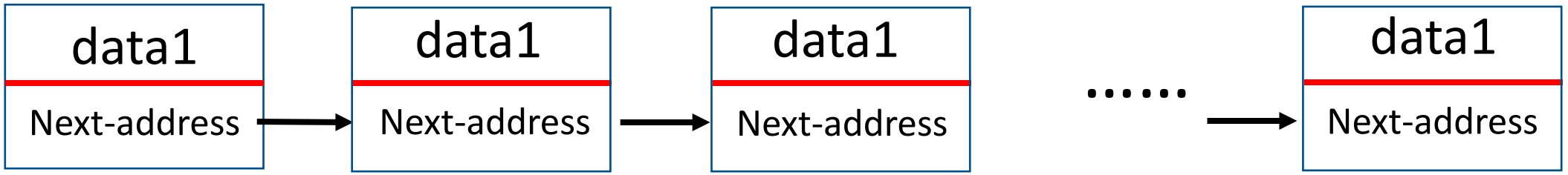
- Linked List -- A data type with dynamic storage.



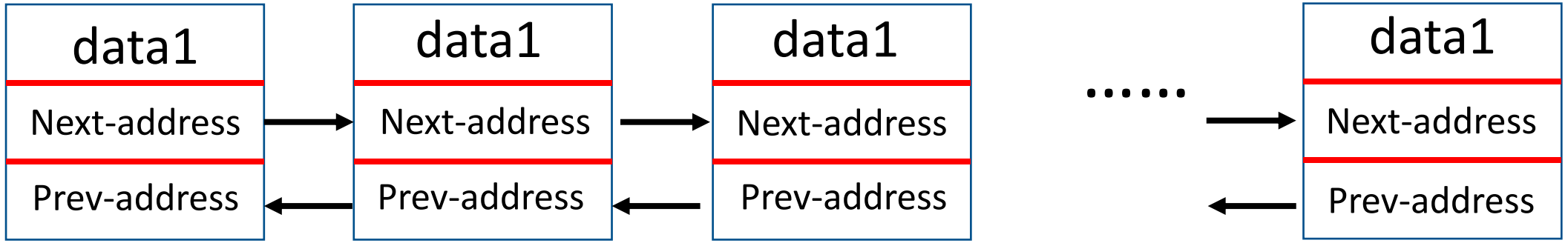
Linked List

- Linked List -- A data type with dynamic storage.

Single linked list (SLL) only has the next-address



Double linked list (DLL) both has the pre-address and the next-address



Linked List

- Node of SLL and DLL

SLL

```
struct node { // type of a node in a single linked list
int data;
struct node * next; // the address of the next node
};
```

DLL

```
struct dNode{ // type of a node in a double linked list
int data;
struct dNode * prev; // address of the previous node
struct dNode * next; // address of the next node
};
```

Remark: The "`prev`" `of the` head node is empty, denoted by NULL (empty address).
The "`next`" `of the tail` node is also an empty address.

3

FUNCTIONS

3.1 Definition of Functions

Function Prototype and Implementation

- Three key components of function prototype.

```
int sum(int x, int y);
```

Return Value Type

Function Name

Parameters

- You have to implement the function prototype be fore you use it.

```
int sum(int x, int y){
    return x + y;
}
```

3.2 Function Parameters

Parameters

- The symbolic name for "data" that passes into a function.

Two ways to pass into a function:

- **Pass by value**
- **Pass by reference**

- Default Parameter values: we can initialize the value of parameters in the prototype. In this case, sometimes we do need to input the value of the parameter, and the function will use the default value.

  Example: sum(int x, int y = 3)

Default Parameters

- Default Parameter values: we can initialize the value of parameters in the prototype. In this case, sometimes we do need to input the value of the parameter, and the function will use the default value.

```
#include <cstring>
#include<iostream>
using namespace std;
int sum(int x, int y=0){
    return x + y;
}
int main()
{
    int x = 2;
    int y = 3;
    cout<<"The sum of x and y is: "<<sum(x,y)<<endl;
    cout<<"The sum of x and y is: "<<sum(x)<<endl;
}
```

```
The sum of x and y is: 5
The sum of x and y is: 2
```

Pass by value: fundamental type

- The parameter is a copy of the original variable

```
int foo(int x)
{   // x is a copy
    x += 10;
    return x;
}

int main()
{
    int num1 = 20;
    int num2 = foo( num1 );

    return 0;
}
```

Will `num1` be changed in foo()?

References as function parameters

- No data copying in the reference version; Better efficiency
- The modification to a reference will affect the original object

```
struct Matrix
{
    int rows;
    int cols;
    float * pData;
};

float matrix_max(struct Matrix mat)
{
    float max = FLT_MIN;
    //find max value of mat
    for(int r = 0; r < mat.rows; r++)
        for (int c = 0; c < mat.cols; c++)
        {
            float val = mat.pData[ r * mat.cols + c];
            max = ( max > val ? max : val);
        }
    return max;
```

```
struct Matrix
{
    int rows;
    int cols;
    float * pData;
};

float matrix_max(struct Matrix & mat)
{
    float max = FLT_MIN;
    //find max value of mat
    for(int r = 0; r < mat.rows; r++)
        for (int c = 0; c < mat.cols; c++)
        {
            float val = mat.pData[ r * mat.cols + c];
            max = ( max > val ? max : val);
        }
    return max;
```

3.3 Return Statement

Return statement

- Statement `return;` is only valid if the function return type is `void`.
- Just finish the execution of the function, no value returned.

```
void print_gender(bool isMale)
{
    if(isMale)
        cout << "Male" << endl;
    else
        cout << "Female" << endl;

    return;
}
```

```
void print_gender(bool isMale)
{
    if(isMale)
        cout << "Male" << endl;
    else
        cout << "Female" << endl;
}
```

3.4 Common Library Functions

Common Library Functions

The header file <cstdlib> provide some common used functions for users.

Function	Description
double atof(const char* str);	converts a byte string to a floating point value
Int abs(int num);	computes absolute value of an integral value
exit()	causes normal program termination with cleaning up
int system(const char* command);	calls the host environment's command processor

<cstdlib> in fact is the header file <stdlib.h> we used in C language.
More functions can refer to https://en.cppreference.com/w/cpp/header/cstdlib

Example

```
#include <iostream>
#include <cstdlib> // Needed to use the exit function
using namespace std;
// Function prototype
void someFunction();
int main ()
{
    someFunction ();
    return 0;
}
void someFunction()
{
    cout << "This program terminates with the exit function. \n";
    cout << "Bye!\n";
    exit (0);
    cout << "This message will never be displayed\n";
    cout << "because the program has already terminated.\n";
}
```

```
This program terminates with the exit function.
Bye!
```

4

CONTROL & LOOP

if and if-else

- Statements are executed conditionally

```
int num = 10;
if (num < 5)
    cout << "The number is less than 5. "
<< endl;
```

```
if (num == 5 )
{
    cout << "The number is 5." << endl;
}
else
    cout << "The number is not 5." << endl;
```

```
if (num < 5)
    cout << "The number is less than 5." <<
endl;
else if (num > 10)
    cout << "The number is greater than 10."
<< endl;
else
    cout << "The number is in range [5,
10]." << endl;
```

while

- Syntax :

```
while( expression )
{
    //...
}
```

- If the condition is true, the statement (loop body) will be executed.

```
int num = 10;
while(num > 0)
{
    cout << "num = " << num << endl;
    num--;
}
```

For

- Syntax

```
for (init-clause; cond-expression; iteration-expression)
    loop-statement
```

- Example

```
int sum = 0;
for(int i = 0; i < 10; i++)
{
    sum += i;
    cout << "Line " << i << endl;
}
cout << "sum = " << sum << endl;
```

```
int sum = 0;
for(int i = 0, int j = 5 ; i < 10; i++)
{
    sum += i+j;
    cout << "Line " << i << endl;
}
cout << "sum = " << sum << endl;
```

C++ allows initialize multiple values!

5

CLASS

5.1 Declaration of Class

Class Declaration Example

```
class Car
{
   public:
      bool AddGas(float gallons);
      float GetMileage();
      // other operations

   private:
      float m_currGallons;
      float m_currMileage;
      // other data
};
```

Class Name

Protection Mechanism

Methods

Protection Mechanism

Data

Class Rules – Coding Standard

- Class names
  - Always begin with capital letter
  - Use mixed case for phrases
  - General word for class (type) of objects
    - Ex: Car, Boat, Building, DVD, List, Customer, BoxOfDVDs, …
- Class data (member variables)
  - Always begin with m_
    - Ex: m_fuel, m_title, m_name, …
- Class operations/methods
  - Always begin with capital letter
    - Ex: AddGas(), Accelerate(), ModifyTitle(), RemoveDVD(), …

Example of Using a Class

```
// Represents a Day of the Year
class DayOfYear
{
      public:
            void Output();
            int m_month;
            int m_day;
};
// Output method - displays a DayOfYear
void DayOfYear::Output()
{
      cout << m_month << "/" << m_day;
}

// Code from main()
            DayOfYear july4th;
            july4th.m_month = 7;
            july4th.m_day = 4;
            july4th.Output();
```

Method Implementation

Scope Resolution Operator: indicates which class this method is from

Class Name

Method Name

```
void DayOfYear::Output()
{
 cout << m_month
        << "/" << m_day;
}
```

Method Body

Separating Classes into Files

```
// Represents a Day of the Year
class DayOfYear
{
  public:
        void Output();
        int m_month;
        int m_day;
};
```

Class Declaration

Goes in file ClassName.h

(DayOfYear.h)

```
// Output method – displays a DayOfYear
void DayOfYear::Output()
{
  cout << m_month << "/" << m_day;
}
```

Class Definition

Goes in file ClassName.cpp

(DayOfYear.cpp)

Using Classes

```
// code from main()
DayOfYear july4th;
july4th.m_month = 7;
july4th.m_day = 4;
july4th.Output();
```

Constructor

Object Name (Variable)

Dot Operator

Class Methods and Members

5.2 Constructor of Class

Constructors

- Special methods that “build” (construct) an object
  - Supply default values
  - Initialize an object

- Automatically called when an object is created
  - implicit: `Date today;`
  - explicit: `Date today(7, 28, 1914);`

Constructor Syntax

- Syntax:
  - For prototype:

```
ClassName();
```

  - For definition:

```
ClassName::ClassName() { /* code */ }
```

- Notice that...
  - There is no return type
  - Same name as class!

Constructor Definition

```
Date::Date (int month, int day,
            int year)
{
    m_month = month;
    m_day = day;
    m_year = year;
}
```

Special Constructor -- *Copy Constructor*

- Just like assign an int variable to another int variable, the copy constructor allow us to assign the object of a class to another object of the same class. It copy attributes of the given object to the attributes of another object.
- The **Copy Constructor** is a special constructor. Thus, its syntax is same as the constructor, but the formal argument should be the reference of an object of the same class.

```
Example(Example & emp);
```

Note, like the default constructor, the copy constructor is also a default member function that the class will automatically create it when you instantiate a class. Generally, you do not need to create it by yourselves.

```
int main(){
    Car Xiaopeng;
    Car LI(3.0,1000.0,100.0);
    Car Weilai(LI);}
```

Call the default constructor.

Call the copy constructor.

Call the self-defined constructor.

5.3 Destructor of Class

Destructors

```
class Student
{
public:
    Student()  // Constructor
    {
        name = new char[1024]{0};
        born = 0;
        male = false;
        cout << "Constructor: Person()"
       << endl;
    }
    ~Student()  // Destructors
    {
        delete [] name;
    }
};
```

- The destructor will be invoked when the object is destroyed.
- Be formed from the class name preceded by a **tilde (~)**
- Have no return value, no parameters
- **Generally!** The *Destructor* is automatically built by complier.

5.4 this Pointer

this pointer

*Every object in C++ has access to its own address through an important pointer called **this** pointer. The **this** pointer is an implicit parameter to all member functions. Therefore, inside a member function, this may be used*

```
#include <iostream>
#include <map>
#include <string>
using namespace std;
class Soilder{
    private:
        int AD;
    public:
        Soilder(int A){AD=A;};
        int awakening(){ AD = 10*AD; }
        int attack(int z){
        this->awakening();
        cout<<"cause harmness<<
"<<z*AD<<endl;
        }
};
```

```
int main()
{
    Soilder hero(3);
    hero.attack(10);
}
```

```
cause harmness<< 300
```

6

INHERITANCE

6.1 Derived Class

Inheritance Relationship Code

```
class Vehicle {
  public:
    // functions
  private:
    int     m_numAxles;
    int     m_numWheels;
    int     m_maxSpeed;
    double m_weight;
    // etc
} ;
```

all Vehicles have axles, wheels, a max speed, and a weight

Inheritance Relationship Code

```
class Car {

} ;
```

Inheritance Relationship Code

```
class Car: public Vehicle {

} ;
```

Car inherits from the Vehicle class

don't forget the colon here!

Inheritance Relationship Code

```
class Car: public Vehicle {
  public:
    // functions
  private:
    int     m_numSeats;
    double  m_MPG;
    string  m_color;
    string  m_fuelType;
    // etc
} ;
```

all Cars have a number of seats, a MPG value, a color, and a fuel type

Inheritance Relationship Code

```
class Car:
  public Vehicle { /*etc*/ };
class Plane:
  public Vehicle { /*etc*/ };
class SpaceShuttle:
  public Vehicle { /*etc*/ };
class BigRig:
  public Vehicle { /*etc*/ };
```

Constructor for Derived Class

```
class Car: public Product, Vehicle{
    private:
        float m_MFG;
    public:
        Car(float MFG,float price,float m_weight);
        void Driving(bool flag);
        void show_NumofWheels();
};
```

```
Car::Car(float MFG,float price,float m_weight): Product(price), Vehicle(m_weight){
    m_MFG = MFG;
};
```

```
Product::Product(float price){   m_price = price;   };
```

```
Vehicle::Vehicle(float weight){  m_weight = weight;  };
```

Note: Constructors of the base classes cannot be inherited. You need to call them in the constructor of the derived class.

7

ENCAPSULATION

7.1 Access Attributes

Derived Class

Inheritance Mode

Base Classes

Derived Class

```
class Car: public Product, Vehicle{
    private:
        float m_MFG;
    public:
        Car(float MFG,float price,float
m_weight);
        void Driving(bool flag);
        void show_NumofWheels();
};
```

Access Attributes:

Public Attribute: Any one inside or outside the class can access this member.

Protected Attribute: Other members inside the class and the derived classes can access the protected member.

Private Attribute: Only members inside the class can access the private member.

Inaccessible Attribute: No one can access the member with inaccessible attribute.

Public Inheritance

The inheritance mode determines the access attributes of the base members in the derived class.

Access Attribute of the Members in Base Class	public	protected	private	Inaccessible
Access Attribute of the base Members in the Derived Class	public	protected	Inaccessible	Inaccessible

Access Attributes:

Public Attribute: Any one inside or outside the class can access this member.
Protected Attribute: Other members inside the class and the derived classes can access the protected member.
Private Attribute: Only members inside the class can access the private member.
Inaccessible Attribute: No one can access the member with inaccessible attribute.

Protected Inheritance

The inheritance mode determines the access attributes of the base members in the derived class.

Access Attribute of the Members in Base Class	public	protected	private	Inaccessible
Access Attribute of the base Members in the Derived Class	protected	protected	Inaccessible	Inaccessible

Access Attributes:

Public Attribute: Any one inside or outside the class can access this member.
Protected Attribute: Other members inside the class and the derived classes can access the protected member.
Private Attribute: Only members inside the class can access the private member.
Inaccessible Attribute: No one can access the member with inaccessible attribute.

```
#include<iostream>
class A{
    public: int T;
    protected: int T2;
    private: int T3;
};

class B: protected A{};
class C: protected B{};
int main(){
    std::cout<<A.T<<std::endl;
    std::cout<<B.T<<std::endl;
    std::cout<<C.T<<std::endl;
}
```

Private Inheritance

The inheritance mode determines the access attributes of the base members in the derived class.

Access Attribute of the Members in Base Class	public	protected	private	Inaccessible
Access Attribute of the base Members in the Derived Class	private	private	Inaccessible	Inaccessible

Access Attributes:

Public Attribute: Any one inside or outside the class can access this member.
Protected Attribute: Other members inside the class and the derived classes can access the protected member.
Private Attribute: Only members inside the class can access the private member.
Inaccessible Attribute: No one can access the member with inaccessible attribute.

8

POLYMORPHISM

8.1 Function Overloading

Function overloading

- Which function to choose? The compiler will perform name lookup.
- Argument-dependent lookup, also known as ADL.
- The return type will not be considered in name lookup.

```
int sum(int x, int y)
{
    cout << "sum(int, int) is called" << endl;
    return x + y;
}
float sum(float x, float y)
{
    cout << "sum(float, float) is called" << endl;
    return x + y;
}
double sum(double x, double y)
{
    cout << "sum(double, double) is called" << endl;
    return x + y;
}
```

- Function names must be the same.
- The parameter lists must be different (different numbers, different types, different order of parameters, etc.)
- The return types of the functions may or may not be the same.
- Only having a different return type is not enough to be an overload of a function.

8.2 Operator Overloading

Operator Overloading

Operator overloading is a form of C++ polymorphism.

To overload an operator, a special form of function called an operator function is used. The format of the operator function is as follows:

operator op(argument-list)

C++ allows operator overloading to be extended to user-defined types, for example allowing + to add two objects.

Remark:

1. op must be a valid C++ operator and cannot invent a new symbol. For example, there cannot be a function like operator@() because there is no @ operator in C++.

2. The overloaded operator could be viewed as a function but we call it using an operator instead of the function name.

Overloading the operator negative "-"

```cpp
#include <iostream>
using namespace std;

class Distance
{
private:
    int feet; int inches;
public:
    // constructor
    Distance() {};
    Distance(int f, int i) {
        feet = f; inches = i;
    };
    // show distance
    void DisplayDistance() {
        cout << "F: " << feet << ", I: " << inches << endl;
    }
    // overloading the opreator ( - )
    Distance operator-() {
        feet = -feet; inches = -inches;
        return Distance(feet, inches);// why need this?
    }
};
```

```cpp
int main(void)
{
    Distance d1(1, 10);
    Distance d2(-5, 110);
    -d1;
    d1.DisplayDistance();
    -d2;
    d2.DisplayDistance();
    return 0;
}
```

8.3 Function Templates

Why Function Templates?

The following code is quit not elegant

```
void swapInt(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

void swapDouble(double &a, double &b)
{
    double temp = a;
    a = b;
    b = temp;
}
```

```
void test01()
{
    int a = 10;
    int b = 20;
    swapInt(a, b);
    cout << "a = " << a << endl;
    cout << "b = " << b << endl;

    double c = 30;
    double d = 40;
    swapDouble(c, d);
    cout << "c = " << c << endl;
    cout << "d = " << d << endl;
}

int main()
{
    test01();
    system("pause");
    return 0;}
```

Why Function Templates?

The two functions looks very very very similar. Can we programing in an easer way?

```
void swapInt(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

void swapDouble(double &a, double &b)
{
    double temp = a;
    a = b;
    b = temp;
}
```

Function Templates:

template<typename T>
Function definition

```
template<typename T>
void swap(T &x, T &y)
{
    T temp = x;
    x = y;
    y = temp;
}
```

Function Templates

Function template can have more than one formal arguments.

template <typename T1, typename T2,....>
Function definition
(Note! T1,T2,... are argument of this funtion)

```
template<typename ITEM,typename Value>
void MyPrint(ITEM x, Value y){
    std::cout<<"Item: "<<x<<"  "<<"Value: "<<y<<std::endl;
}
```

```
int main(){
    MyPrint(9527,1000);
    MyPrint("TBH",5000);
    return 0;
}
```

```
Item: 9527  Value: 1000
Item: TBH  Value: 5000
```

8.4 Class Templates

Class Templates

Class Templates: to create a general class, the member data types in the class can not be specified, and a virtual type is used to represent.

template<typename T1, typename T2,...>

Class Definition

```
template<typename Data>
class AlgebraicOperation
{
private:
    Data data1;
    Data data2;
public:
    Person(Data A, Data B)
    {
        data1 = A;
        data2 = B;
    }
    Data sub()
    {
        return data1-data2;
    }
    Data add()
    {
        return data1+data2;
    }
    Data mul()
    {
        return data1*data2;
    }
    Data div()
    {
        return data1/data2;
    }
};
```

Class Templates

Usage of Class Templates:

```
ClassTemplateName<Datetype> ObjectName(T1,T2);
```

Remark: Class templates does not have automatic type deduction. Thus, you need to specify the datatype when you use class templates.

Example:

```
AlgebraicOperation<int> int_type(3,4);
AlgebraicOperation int_type(T1,T2); // wrong!
```

Class Templates

The member functions in a class template are function templates.

```
template<typename Data>
class AlgebraicOperation
{
private:
    Data data1;
    Data data2;
public:
    Person(Data A, Data B)
    {
        data1 = A;
        data2 = B;
    }
    Data sub()
    {
        return data1-data2;
    }
    Data add()
    {
        return data1+data2;
    }
    Data mul()
    {
        return data1*data2;
    }
    Data div()
    {
        return data1/data2;
    }
};
```

Member Function of Class Templates

The member functions in a class template are function templates. You should define them in the syntax of template functions.

```
template<class T1, class T2>
class Person
{
private:
    T1 m_Name;
    T2 m_Age;
public:
    Person(T1 name, T2 age);
    void showPerson();
};

template<class T1,class T2>
Person<T1, T2>::Person(T1 name, T2 age)
{
    this->m_Name = name;
    this->m_Age = age;
}
```

```
template<class T1, class T2>
void Person<T1, T2>::showPerson()
\\ void Person::showPerson() normal class.
{
    cout << “Name:" << m_Name << " age:" <<
m_Age << endl;
}

int main()
{
    Person<string, int>p("Tom", 30);
    p.showPerson();
    return 0;
}
```

9

STANDARD TEMPLATE LIBRARY (STL)

Standard Template Library (STL)

The Standard Template Library (STL) is a software library originally designed by Alexander Stepanov for the C++ programming language that influenced many parts of the C++ Standard Library. It provides four components called *algorithms*, *containers*, *functions*, and *iterators*.

The STL provides a set of common classes (class templates) for C++ that can be used with any built-in type and with any user-defined type that supports some elementary operations (such as copying and assignment).

Example:

The container *Vector* in the header file ” vector”.

Vector

The usage of Vector in the header file <vector>

```
int main()
{
    vector<int> vec1;
    vector<float> vec2(3);
    vector<char> vec3(3,'a');
    vector<char> vec4(vec3);
    return 0;
}
```

Remark: As can be seen from the usage, "*Vector* " is noting but a class template.

```
vector<char> vec4(vec3);
```

Class Template Name

Specification of the datatype

Object of the Class Template

Vector

Usage of member functions of Vector

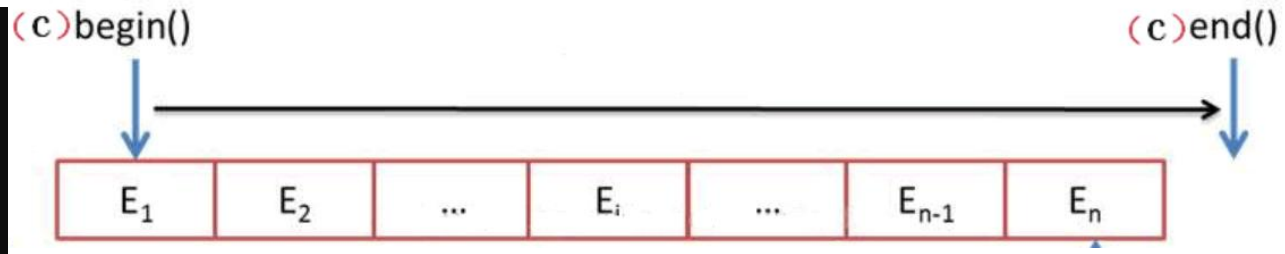
Member Function	**Description**	**Usage**
empty()	Check whether this vector is empty.	`vector<int> vec1; vec1.empty()`
size()	Get the size of this vector	`vector<int> vec1; vec1.size()`
vec1[i] (**operation overloading**)	Get the i-th element in the vector	`vector<int> vec1; vec1[i]`
push_back(x)	Add an element x into the end of the vector	`vector<int> vec1; vec1. push_back(x)`

More member functions:
https://blog.csdn.net/qq_31112171/article/details/127720625

Iteration of Vector

Vector has two member functions begin() and end(), which point to the head pointer and the tail pointer of the vector. Thus, we can use pointer operations to loop elements in the vector.

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
vector<int>values({1,2,3,4,5});
auto first = values.begin();
auto end = values.end();
while (first != end)
{
    cout << *first << " ";
    ++first;
}
return 0;
}
```




```
for (auto first = values.begin(); first !=
values.end(); ++first) {
    cout << *first << " ";
}
```

```
int main()
{
    vector<int>values{1,2,3,4,5};
    for(int i; i<values.size();i++){
        cout<<values[i];
    }
```

Also OK

map

The usage of map in the header file <map>.

```
map<datatype 1,datatype 2> my_map;
```

It implies map the datatype1 to the datatype 2. For example,

```
#include <iostream>
#include <map>
#include <string>
using namespace std;
int main()
{
    map<string,int> map1;
    string j = "joey";
    map1[j] = 12586;
    cout<<"The id of"<<j<<" is: "<<map1[j];
}
```

```
map<string,int> map1
```

Class Template Name

Specification of the datatype

Object of the Class Template

map

Usage of member functions of map

Member Function	Description
my_map.find()	Find a element in the map
my_map.size()	Get the size of this map
my_map.begin()	Retrun the head pointer of the map
my_map.end()	Retrun the tail pointer of the map

More member functions: https://blog.csdn.net/weixin_41501074/article/details/114532738

10

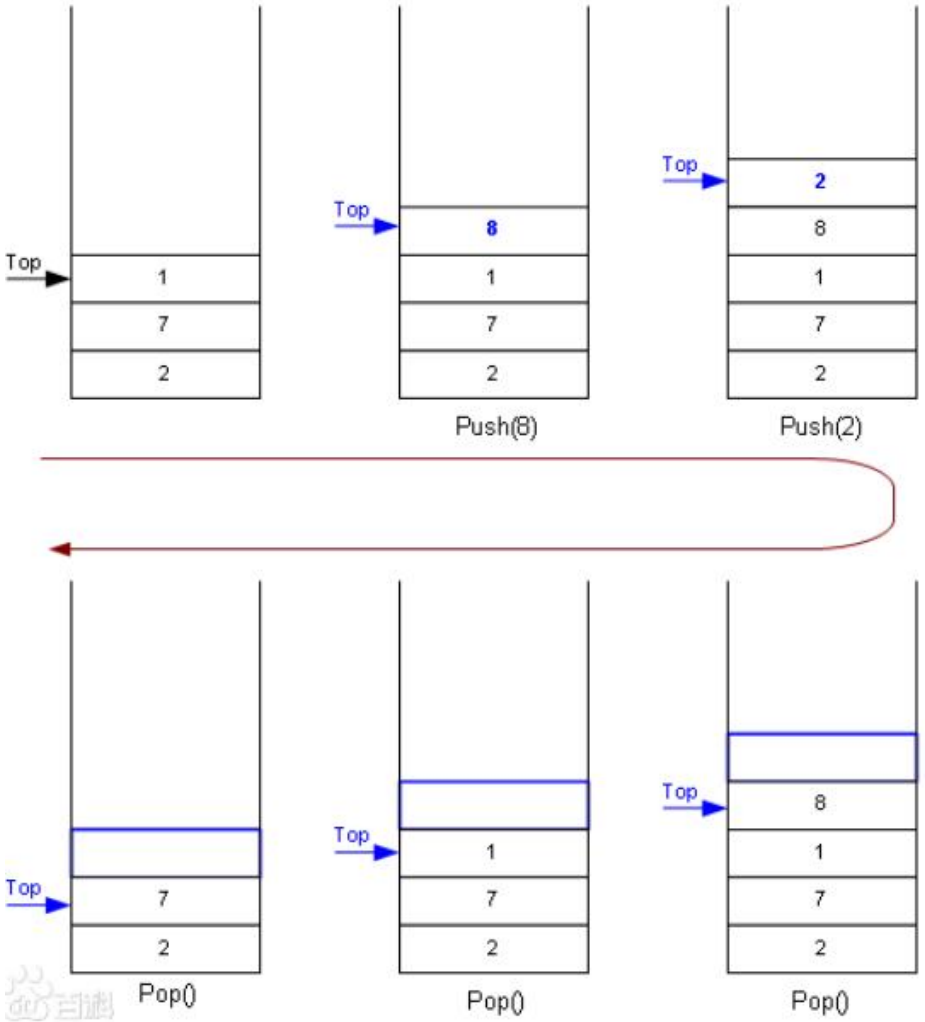
MEMORY MODULE

10.1 Automatic Storage

Automatic Storage

Automatic Storage:

- Ordinary variables defined inside a function use automatic storage and are called automatic variables
- They expire when the function terminates
- Automatic variables typically are stored on a stack
- A last-in, first-out, or LIFO, process



Stack: last-in first-out

Automatic Storage

```
#include<iostream>
using namespace std;
int* get(){
    int x = 4;
    return &x;
}
int main(){
    int* z = get();
    cout<<z<<endl;
    cout<<*z<<endl;
}
```

The value of z is 0x0 since the address of x has been released. Naturally, the operation “*z” is invalid.

10.2 Static Storage

Static Storage

Static Storage

- Static storage is storage that exists throughout the execution of an entire program
- Two ways
  1. Define it externally, outside a function
  2. Use the keyword static when declaring a variable : *static int x = 3;*

Static Storage

```
#include<iostream>
using namespace std;
int* get(){
    int x = 4;
    return &x;
}

int main(){
    int* z = get();
    cout<<z<<endl;
    cout<<*z<<endl;
}
```

Automatic Storage

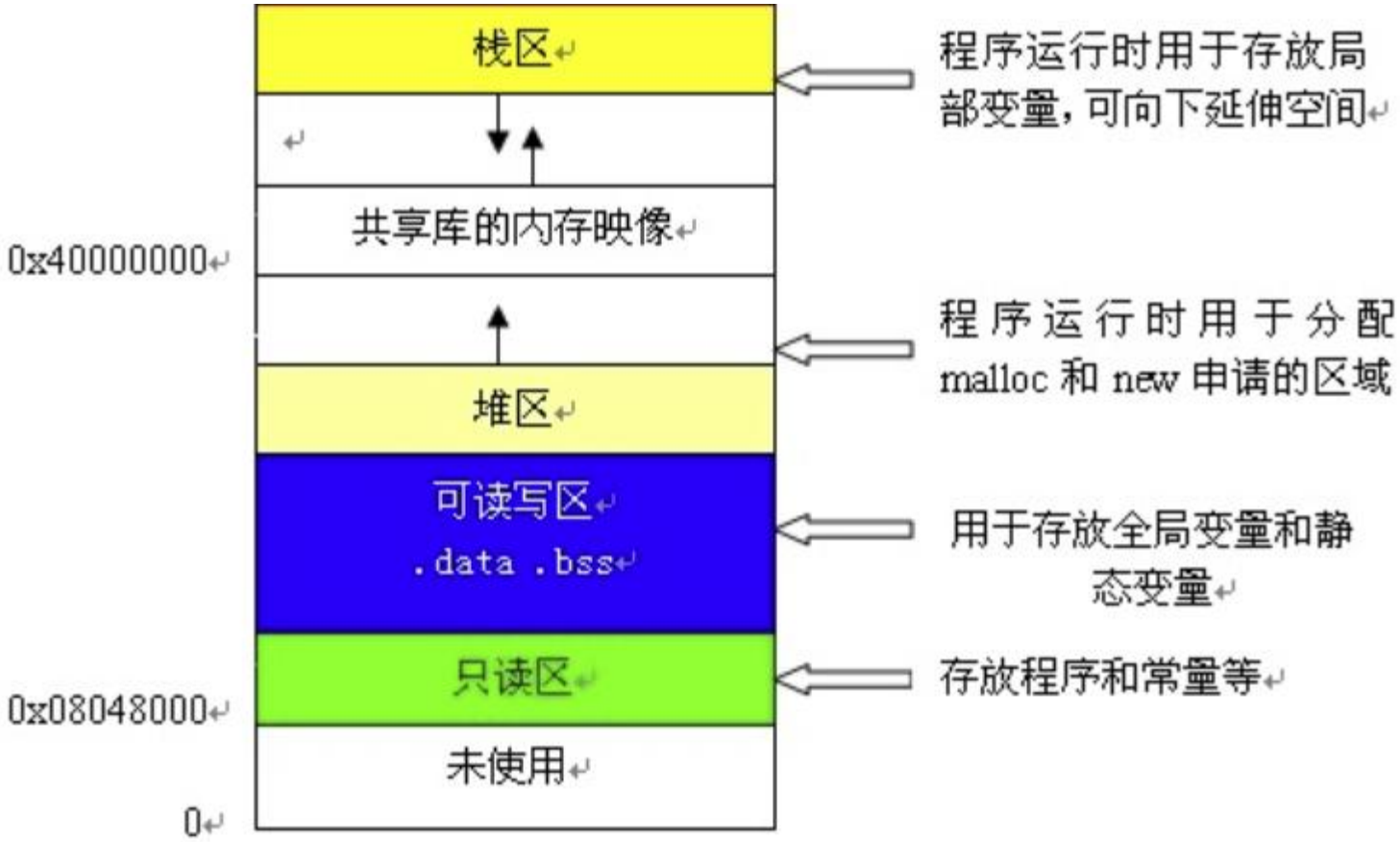
```
#include<iostream>
using namespace std;
int* get(){
    static int x = 4;
    return &x;
}

int main(){
    int* z = get();
    cout<<z<<endl;
    cout<<*z<<endl;
}
```

Static Storage

The value of z is 0x403010 since the address of x still exists. Naturally, the operation “*z” is valid and the value is 4.

10.3 Dynamic Storage

Heap and Stack



Heap: 堆

Stack: 栈

Dynamic Storage

Dynamic Storage

- The new and delete operators provide a more flexible approach than automatic and static variables.
- Refer to as the free store or heap
- Lifetime of the data is not tied arbitrarily to the life of the program or the life of a function

Remark: The operators new and delete should appear in pairs.

Dynamic Storage

```
#include<iostream>
using namespace std;
int* get(int *x){
    cout<<"In get, the address x is "<<x<<" and its value is "<<*x<<endl;
    return x;
}
int main(){
    int *x = new(int);
    *x = 3;
    x = get(x);
    cout<<"In main, the address x is "<<x<<" and its value is "<<*x<<endl;
    delete x;
    cout<<"After delete, the address x is "<<x<<" and its value is "<<*x<<endl;
}
```

```
In get, the address x is 0x1e1a80 and its value is 3
In main, the address x is 0x1e1a80 and its value is 3
After delete, the address x is 0x1e1a80 and its value is 1973936
```

Allocating Memory with new

In C++, we use new

- Tell new for what data type you want memory
- Let new find a block of the correct size
- Return the address of the block
- Assign this address to a pointer
- This is an example:

```
int * ptr_int = new int; *ptr_int = 1;
```

Freeing Memory with delete

delete operator enables you to return memory to the memory pool

- The memory can then be reused by other parts of the program
- Cannot free a block of memory that you have previously freed
- Cannot use delete to free memory created by ordinary variable

```
int *x = new(int);
delete x; \\ ok
delete x; \\ not ok
int y = 5;
int *prt_y = &y;
delete prt_y; \\ not ok
```

Using new to Create Dynamic Arrays

Use new with larger chunks of data, such as arrays, strings, and structures

Tell new for what data type you want memory

- Static binding: the array is built into the program at compile time
- Dynamic binding: the array is created during runtime

```
int y = 20;
int *my_array = new int[y];
cout<<*my_array;
delete [] my_array;
```

Remark:

1. Use delete [] if you used new [] to allocate an array
2. Use delete (no brackets) if you used new to allocate a single entity

Using new to Create Dynamic Arrays

Use new with larger chunks of data, such as arrays, strings, and structures

Tell new for what data type you want memory

- Static binding: the array is built into the program at compile time
- Dynamic binding: the array is created during runtime

```
int y = 20;
int *my_array = new int[y];
cout<<*my_array;
delete [] my_array;
```

Remark:

1. Use delete [] if you used new [] to allocate an array
2. Use delete (no brackets) if you used new to allocate a single entity

11

SCOPES & NAMESPACE

11.1 Scopes

Scopes

- Named entities, such as variables, functions, and compound types need to be declared before being used in C++.
- An entity declared outside any block has global scope, meaning that its name is valid anywhere in the code.
- An entity declared within a block, such as a function or a selective statement, has block scope, and is only visible within the specific block in which it is declared, but not outside it.
- Local variables have block scope.

Scopes

```
int foo; // global variable
int some_function ()
{
 int bar; // local variable
 bar = 0;
}
int other_function ()
{
 foo = 1; // ok: foo is a global variable
 bar = 2; // wrong: bar is not visible from this
function
}
```

Scopes

- In each scope, a name can only represent one entity. For example, there cannot be two variables with the same name in the same scope:

```
int some_function ()
{
 int x;
 x = 0;
 double x; // wrong: name already used in this scope
 x = 0.0;
}
```

Remark: Name is to tied with the data type. In other words, int x and float x have the same name.

Variables

Variables have **two major attributes** in their lifetime:

- **Memory Module**, such static, automatic, and adynamic, determines when the variables will expire.
- **Scopes**, such as global and block scope, determines which part of the program can use these variables.

11.2 Namepspace

Namespaces

Namespaces provide us a flexible way to using variables globally. This allows organizing the elements of programs into different logical scopes referred to by names.

```
namespace identifier
{
 named_entities
}
```

```
// namespaces
#include <iostream>
namespace foo
{
 int value() { return 5; }
}
namespace bar
{
 const double pi = 3.1416;
 double value() { return 2*pi; }
}
int main () {
 std::cout << foo::value() << '\n';
 std:: cout << bar::value() << '\n';
 std:: cout << bar::pi << '\n';
 return 0;
}
```

Namespaces

Namespaces can be split: Two segments of a code can be declared in the same namespace:

```
namespace foo { int a; }
namespace bar { int b; }
namespace foo { int c; }
```

Namespaces

The keyword using introduces a name into the current declarative region (such as a block), thus avoiding the need to qualify the name. For example:

```
#include <iostream>
using namespace std;
namespace first
{ int x = 5; int y = 10; }
namespace second
{ double x = 3.1416; double y = 2.7183; }
int main () {
 using first::x;
 using second::y;
 cout << x << '\n';
 cout << y << '\n';
 cout << first::y << '\n';
 cout << second::x << '\n';
 return 0;
}
```

Namespaces

The keyword using can also be used as a directive to introduce an entire namespace.

```
// using
#include <iostream>
using namespace std;
namespace first
{ int x = 5; int y = 10; }
namespace second
{ double x = 3.1416; double y = 2.7183; }
int main () {
 using namespace first;
 cout << x << '\n';
 cout << y << '\n';
 cout << second::x << '\n’;
 cout << second::y << '\n';
 return 0;
}
```

Namespace Aliasing

Existing namespaces can be aliased with new names, with the following syntax:

```
namespace new_name = current_name;
```

The std Namespace

All the entities (variables, types, constants, and functions) of the standard C++ library are declared within the std namespace.

```
std::cout
std::cin
std::string
```

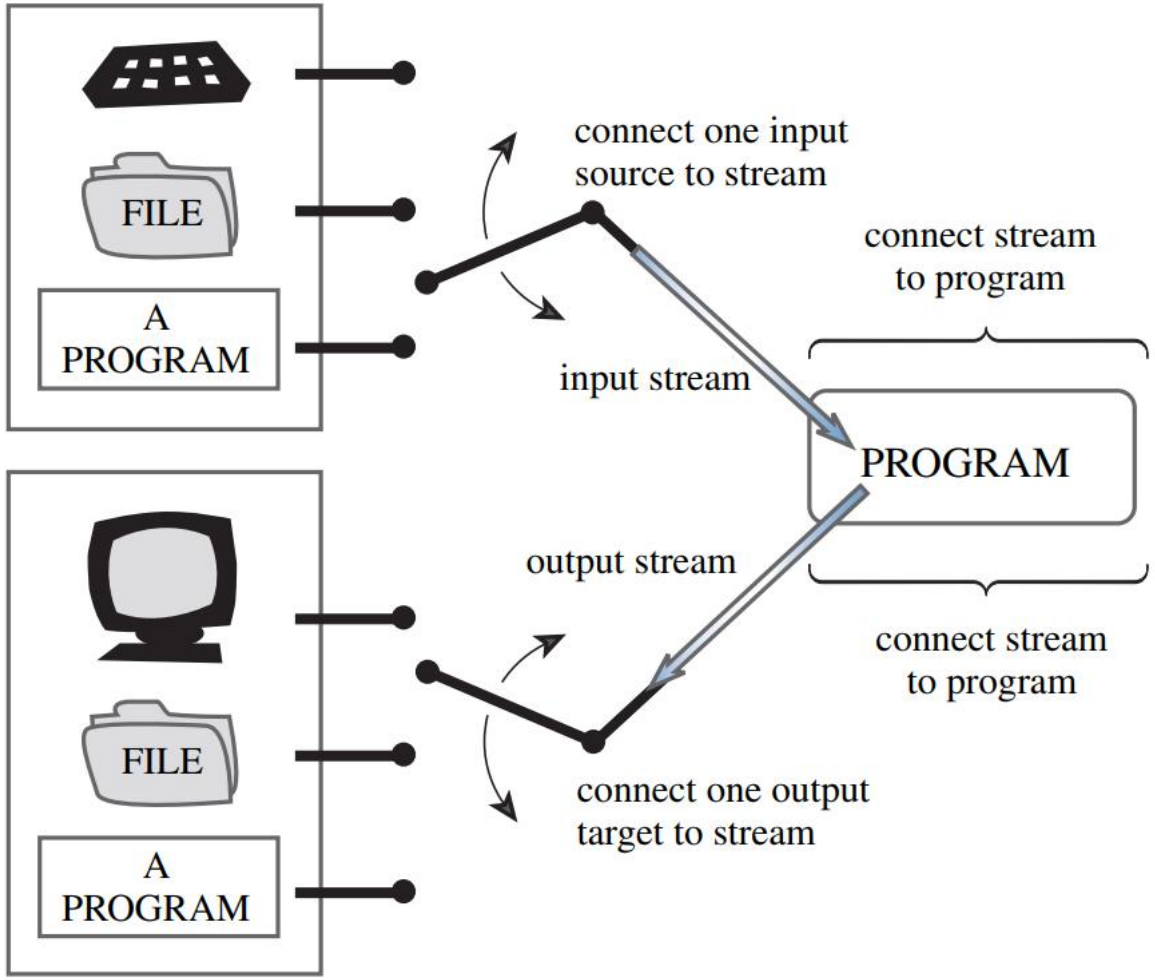
12

I/O Streams

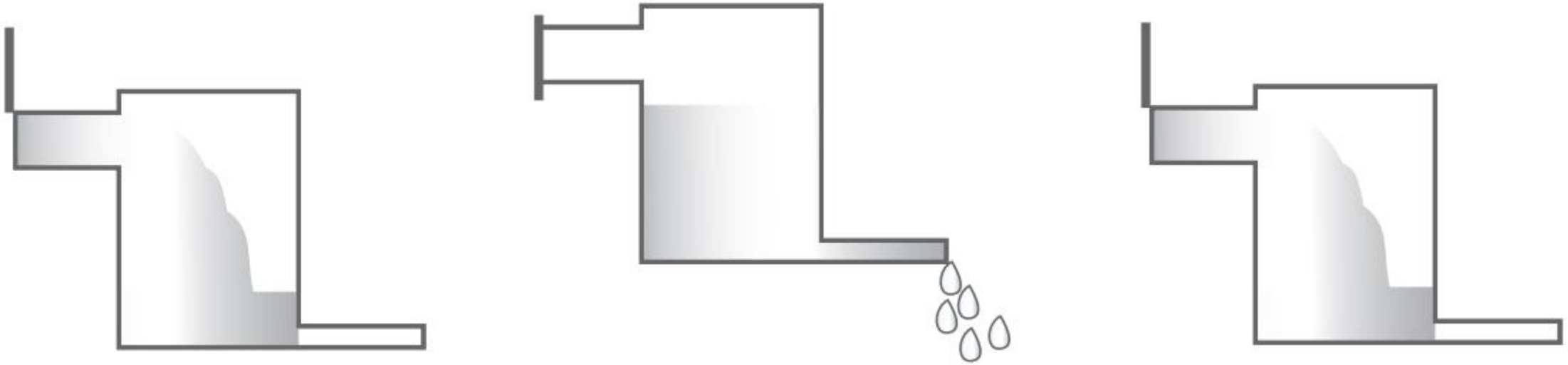
12.1 What are Streams?

Stream

A C++ program views input or output as a stream of bytes. On input, a program extracts bytes from an input stream, and on output, a program inserts bytes into the output stream. Each byte can represent a character. The bytes can form a binary representation of character or numeric data. The bytes in an input stream can come from the keyboard, but they can also come from a storage device, such as a hard disk, or from another program. Similarly, the bytes in an output stream can flow to the display, to a printer, to a



Buffer



fill stream buffer with block of data stream outflow feeds program byte-by-byte refill stream buffer with next block of data

Usually, input and output can be handled more efficiently by using a buffer. A ***buffer*** is a block of memory used as an intermediate, temporary storage facility for the transfer of information from a device to a program or from a program to a device

Buffered keyboard input allows the user to back up and correct input before transmitting it to a program. A C++ program normally flushes the input buffer when you press Enter. That's why programs in our course don't begin processing

12.2 Standard I/O Streams

I/O Streams

- `iostream` library:
  - `<iostream.h>`: Contains `cin`, `cout`, `cerr` and `clog` objects
- `ios`:
  - `istream` class and `ostream` class inherit from `ios` base class
    - `iostream` inherits from `istream` and `ostream`.
- << (left-shift operator)
  - Overloaded as *stream insertion operator*
- >> (right-shift operator)
  - *Overloaded as stream extraction operator*
- Both operators used with cin, cout, cerr, clog, and with user-defined stream objects

I/O Streams

- `iostream` library:
  - `<iostream.h>`: Contains `cin`, `cout`, `cerr` and `clog` objects
- `ios`:
  - `istream` class and `ostream` class inherit from `ios` base class
    - `iostream` inherits from `istream` and `ostream`.
- << (left-shift operator)
  - Overloaded as *stream insertion operator*
- >> (right-shift operator)
  - *Overloaded as stream extraction operator*
- Both operators used with cin, cout, cerr, clog, and with user-defined stream objects

I/O Streams

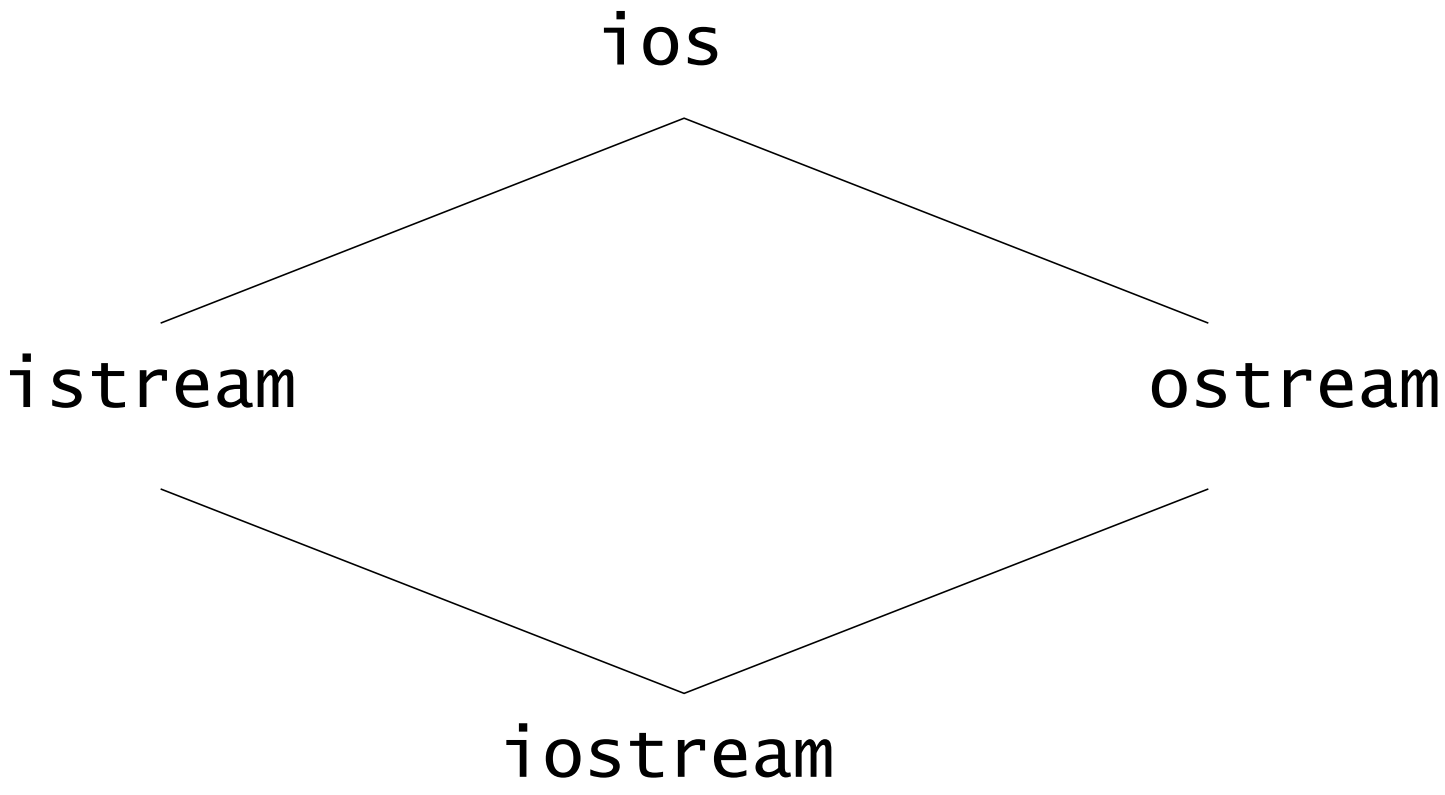


Figure 21.1 Portion of the stream I/O class hierarchy.

12.3 Output Streams

Stream-Insertion Operator: “<<“

- `cout` is an object of the output stream “ostream”.
- << is overloaded to output built-in types
  - `cout << ‘\n’;`
    - Prints newline character
  - `cout << endl;`
    - `endl` is a stream manipulator that issues a newline character. Newline but without the character ‘\n’.

Stream-Insertion Operator: “<<“

<< : Associates from left to right, and returns a reference to its left-operand object (i.e. `cout`).

- This enables cascading

```
cout << "How" << " are" << " you?";
```

- Make sure to use parenthesis:

```
cout << "1 + 2 = " << (1 + 2);   (Correct)

cout << "1 + 2 = " << 1 + 2;     (Wrong)
```

Change the output format of cout

The following statements tell the cout to change the format of the output numbers.

cout<<hex; cout<<oct; cout<<dec;

The operation “cout<<hex” also will return the reference of “cout”, but the output format has been changed to hexadecimal mode.

```
int main()
{
    int x = 15;
    cout<<"The hexadecimal form of the x = "<<x<<" is ";
    cout<<hex; // change cout to hexadecimal mode. No bytes be casted in to
the bytes stream “cout”.
    cout<<x<<endl;
    cout<<dec; // change cout back to decimal mode.
    cout<<"The octal form of the x = "<<x<<" is ";
    cout<<oct; // change cout to octal mode.
    cout<<x;
}
```

```
The hexadecimal form of the x = 15 is f
The octal form of the x = 15 is 17
```

Example of cout.put()

```
#include<iostream>
using namespace std;

int main(){
    int A = 65;
    cout<<"The character corresponding to the ASCII code: "<<A<<" is ";
    cout.put(A);
}
```

Other Member Functions of cout

- The write() method writes an entire string and has the following template prototype:

```
basic_ostream<charT,traits>& write(const char_type* s, streamsize n);
```

- The first argument to write() provides the address of the string to be displayed.
- The second argument indicates how many characters to display.
- The return type is ostream &.

```
#include <iostream>
using namespace std;
int main(){
    cout.write("I love basketball",5)<<" too hard";
}
```

12.4 Input Streams

Stream-Extraction Operator: “>>”

- cin is an object of the input stream “istream”.

- >> (stream-extraction)
  1. Used to perform stream input
  2. Normally ignores whitespaces (spaces, tabs, newlines)
  3. Returns zero (false) when EOF is encountered, otherwise returns reference to the object from which it was invoked (i.e. cin)
  4. Popular way to perform loops

```
while (cin >> grade)
```

Extraction returns 0 (`false`) when EOF encountered, and loop ends

Other Member Functions of cin

- `cin.eof()`: returns `true` if end-of-file has occurred on `cin`
- `cin.get()`: inputs a character from stream (even white spaces) and returns it
- `cin.get(c)`: inputs a character from stream and stores it in `c`

Example

```
#include <iostream>
using std::cout;
using std::cin;
using std::endl;
int main()
{
    char c;
    cout<<"\nEnter a sentence followed by end-of-file:\n";
    while ( ( c = cin.get() ) != EOF || !cin.eof())
        cout.put( c );
    cout << "\nEOF in this system is: " << c;
    cout << "\nAfter input, cin.eof() is " << cin.eof() << endl;
    return 0;
} // end function main
```

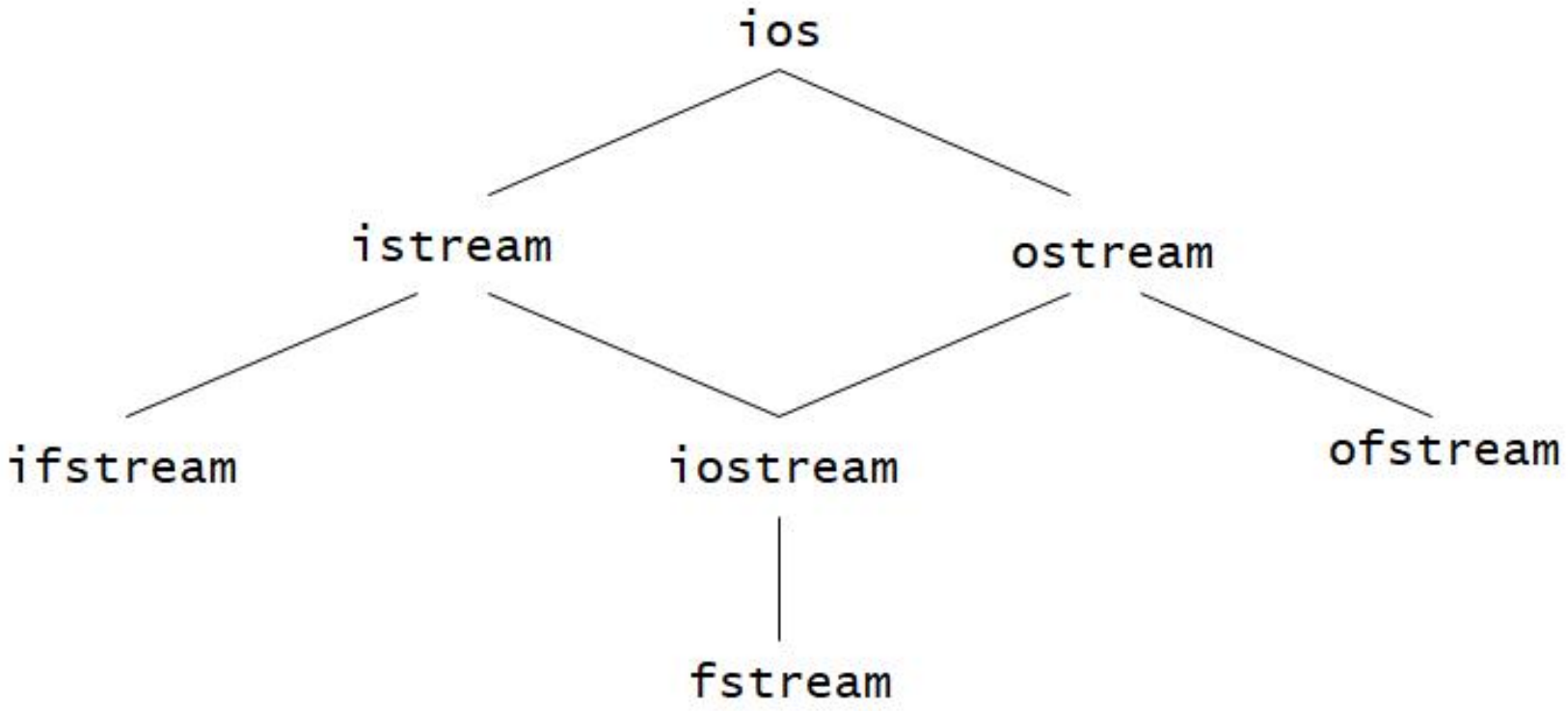
12.5 File I/O Streams

What are files of a computer?

A computer file

1. is stored on a secondary storage device (e.g., disk);
2. is permanent;
3. can be used to provide input data to a program, or receive output data from a program or both;
4. should reside in Project directory for easy access;
5. must be opened before it is used.

File streams



file stream (fstream)

- **ifstream** - defines new input stream (normally associated with a file).
- **ofstream** - defines new output stream (normally associated with a file).

General File I/O Steps

1. Include the header file fstream in the program.

   (Note: There is no ".h" on standard header files : <fstream>)

1. Declare file stream variables.

2. Associate the file stream variables with the input/output sources.

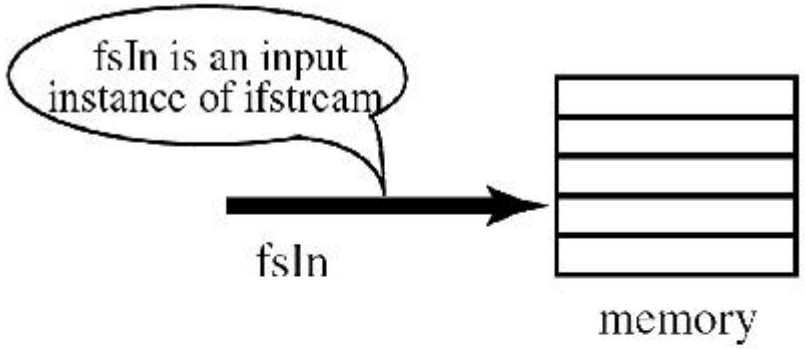
3. Open the file

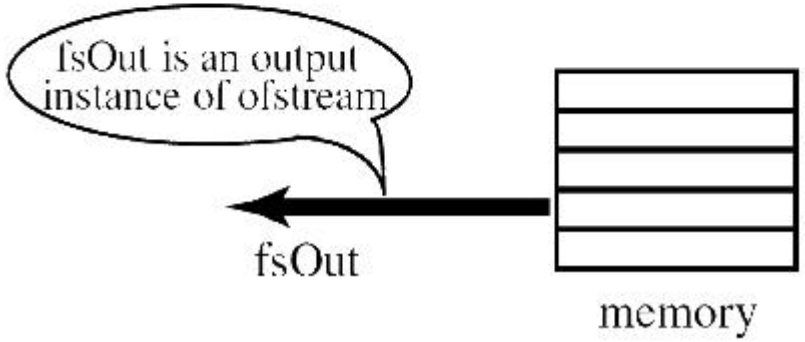
4. Use the file stream variables with >>, <<, or other input/output functions.

5. Close the file.

Simple Example of File Streams

```
#include <fstream>
using namespace std;

int main ()
{ /* Declare file stream variables such as
 the following  */
    ifstream  fsIn;//input
    ofstream fsOut; // output
    //Open the files
    fsIn.open("input.txt"); //open the input
file
    fsOut.open("output.txt"); //open the output
file
    //Code for data manipulation
    //Close files
    fsIn.close();
    fsOut.close();
    return 0;
}
```





Member Functions of File streams: "Open()"

- Opening a file associates a file stream variable declared in the program with a physical file at the source, such as a disk.

- In the case of an input file:
  1. the file must exist before the open statement executes.
  2. If the file does not exist, the open statement fails and the input stream enters the fail state.

- An output file does not have to exist before it is opened;
  1. If the output file does not exist, the computer prepares an empty file for output. 2. If the designated output file already exists, by default, the old contents are erased when the file is opened.

open()

```
#include <fstream>
using namespace std;

int main ()
{ /* Declare file stream variables such as
 the following  */
    ifstream  fsIn;//input
    ofstream fsOut; // output
    //Open the files

    fsIn.open("input.txt"); //open the input file

    fsOut.open("output.txt"); //open the output file
    //Code for data manipulation
    //Close files
    fsIn.close();
    fsOut.close();
    return 0;
}
```

Validate the file before trying to access

Second method

By using bool is_open() function.

If (! Mystream.is_open()) {

Cout << “File is not open.\n ”;

}

```
#include<fstream>
#include<iostream>
using namespace std;

int main(){
    ifstream outFile;
    outFile.open("input.txt");
      // Open validation
    if(!outFile.is_open()) {
        cout << "cannot open file.\n";
        return 1;
    }
        return 0;
}
```

File I/O Example: Reading

Read char by char

```
#include <iostream>
#include <fstream>
int main()
{//Declare and open a text file
ifstream openFile(“data.txt");
 char ch;
 //do until the end of file
while( ! OpenFile.eof() )
{
OpenFile.get(ch); // get one character
cout << ch;   // display the character
}
OpenFile.close(); // close the file

    return 0;
}
```

Read a line

```
#include <iostream>
#include <fstream>
#include <string>
int main()
{//Declare and open a text file
ifstream openFile("data.txt");
string line;
while(!openFile.eof())
{//fetch line from data.txt and put it in a string
getline(openFile, line);
cout << line;
}
openFile.close(); // close the file
    return 0; }
```

More Ouput File-Related Functions

- **ofstream fsOut;**
- **fsOut.open(const char[] fname)**
  - connects stream **fsOut** to the external file ***fname.***
- **fsOut.put(char character)**
  - inserts character ***character*** to the output stream **fsOut.**
- **fsOut.eof()**
  - tests for the end-of-file condition.

File I/O Example: Writing

use the constructor

```
#include <fstream>
using namespace std;
int main()
{/* declare and automatically open the file*/
ofstream outFile("fout.txt");

//behave just like cout, put the word into  the file
outFile << "Hello World!";
outFile.close();
return 0;
}
```

use Open function

```
#include <fstream>
using namespace std;
int main()
{// declare output file variable
ofstream outFile;
// open an exist file fout.txt
    outFile.open("fout.txt”);
//behave just like cout, put the word into  the file
outFile << "Hello World!";
outFile.close();
return 0;
}
```

File Open Mode

Name	Description
ios::in	Open file to read
ios::out	Open file to write
ios::app	All the data you write, is put at the end of the file.
ios::ate	All the data you write, is put at the end of the file. It does not call ios::out
ios::trunc	Deletes all previous content in the file. (empties the file)
ios::nocreate	If the file does not exists, opening it with the open() function gets impossible.
ios::noreplace	If the file exists, trying to open it with the open() function, returns an error.
ios::binary	Opens the file in binary mode.

The red colored part could be refered to *https://cplusplus.com/doc/tutorial/files/*

File Open Mode

```
#include <fstream>
int main(void)
{
ofstream outFile("file1.txt", ios::out);
outFile << "That's new!\n";
outFile.close();
        Return 0;
}
```

If you want to set more than one open mode, just use the **OR** operator- **|**. Like this way:

ios::ate | ios::binary

Example Reading from file

```
#include <iostream>
#include <fstream>
#include <string>
#include <sstream>
using namespace std;
void main()
{ ifstream INFile("number.txt"); //Declare and open a text file
    string line;  int total=0;
    while(! INFile.eof())
    {
    getline(INFile, line); //converting line string to int
    stringstream(line) >> total;
    cout << line  <<endl;
    cout <<total +1<<endl;}
    INFile.close(); // close the file
 }
```

Example writing to file

```
#include <iostream>
#include <fstream>
using namespace std;
void  main()
{
ofstream outFile;
outFile.open("number.txt",ios::app); // open an exist file fout.txt
if (!outFile.is_open())
{ cout << " problem with opening the file ";}
else
{outFile <<200 <<endl ;
cout << "done writing" <<endl;}
outFile.close();
}
```

13

MULTIPLE FILES PROGRAMING

A Program with Multiple Files

MainTime.cpp

```
// usetime0.cpp -- using the first draft of the
Time class
// compile usetime0.cpp and mytime0.cpp
together
#include <iostream>
#include "Time.h"

int main()
{
using std::cout;
using std::endl;
Time planning;
Time coding(2, 40);
Time fixing(5, 55);
Time total;
cout << "planning time = ";
planning.Show();
cout << endl;
cout << "coding time = ";
```

```
coding.Show();
cout << endl;
cout << "fixing time = ";
fixing.Show();
cout << endl;
total = coding.Sum(fixing);
cout << "coding.Sum(fixing) = ";
total.Show();
cout << endl;
return 0;
}
```

A Program with Multiple Files

Time.h

```
class Time
{
    private:
        int hours;
        int minutes;
    public:
        Time();
        Time(int h, int m = 0);




        void Show() const;
};
```

A Program with Multiple Files

```
#include <iostream>
#include "Time.h"
Time::Time()
{
hours = minutes = 0;
}
Time::Time(int h, int m )
{
hours = h;
minutes = m;
}
void Time::AddMin(int m)
{
minutes += m;
hours += minutes / 60;
minutes %= 60;
}
void Time::AddHr(int h)
{
hours += h;
}
```

Time.cpp

```
void Time::Reset(int h, int m)
{
hours = h;
minutes = m;
}
Time Time::Sum(const Time & t) const
{
Time sum;
sum.minutes = minutes + t.minutes;
sum.hours = hours + t.hours + sum.minutes / 60;
sum.minutes %= 60;
return sum;
}
void Time::Show() const
{
std::cout << hours << "hours"<< minutes <<
"minutes";
}
```

Compile Multiple Files

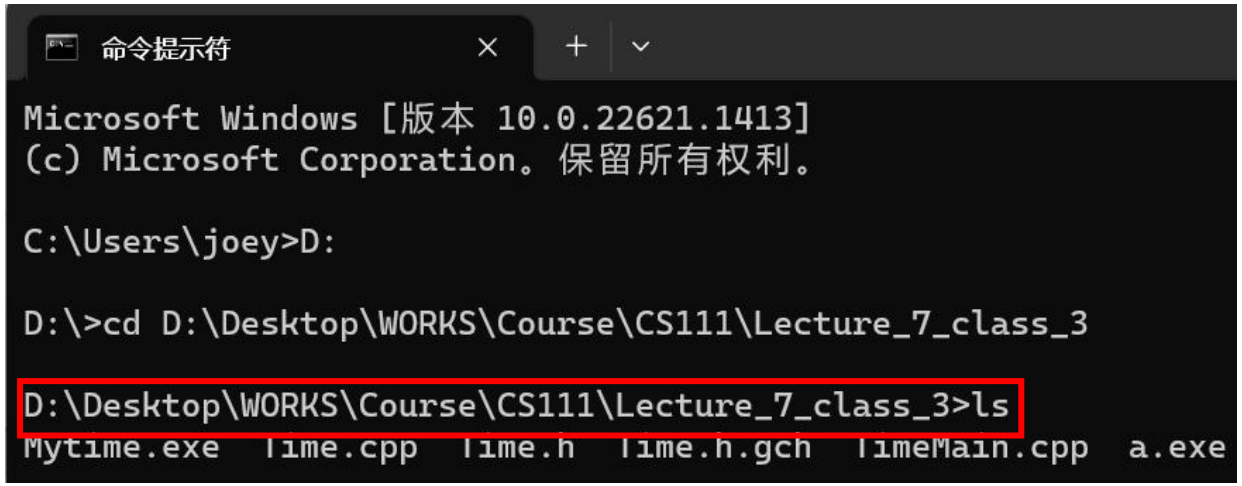
1. Directly Using Compiler g++:

step 1: open terminal;

step 2: change current folder to the folder of your C++ files, using the command "cd path of the target folder".

*(**Note**: 1. if your C++ files are located at the disk other than C, you should use the command "D: " to change the disk from C to others, e.g. D*

2. You can use the command "dir" or "ls" to browse all files under the current folder)



The path before the notation ">" is your current path.

Compile Multiple Files

step 3: compile all the files using the command

"g++ file1.cpp file2.h file3.h …. fileN.cpp –o Name of the exe file"

```
命令提示符

Microsoft Windows [版本 10.0.22621.1413]
(c) Microsoft Corporation。保留所有权利。

C:\Users\joey>D:

D:\>cd D:\Desktop\WORKS\Course\CS111\Lecture_7_class_3

D:\Desktop\WORKS\Course\CS111\Lecture_7_class_3>ls
Mytime.exe  Time.cpp  Time.h  Time.h.gch  TimeMain.cpp  a.exe

D:\Desktop\WORKS\Course\CS111\Lecture_7_class_3>g++ Time.cpp Time.h TimeMain.cpp -o Mytime.exe

D:\Desktop\WORKS\Course\CS111\Lecture_7_class_3>
```

Compile Multiple Files

2. Using Vscode:

You should modify your configuration file ***"task.json"*** in the folder ***".vscode"*** as follows.

tasks.json

.vscode > tasks.json > [ ] tasks > {} 0 > [ ] args > 2

```
1  {
2      "tasks": [
3          {
4              "type": "cppbuild",
5              "label": "C/C++: g+
6              "command": "C:\\Use
7              "args": [
8                  "-fdiagnostics-
9                  "-g",
10                 "${file}",
11                 "-o",
12                 "${fileDirname}
13             ],
```

This setting means that your complier can only **run the current file**, typically is the main file.

tasks.json

.vscode > tasks.json > [ ] tasks > {} 0 > [ ] args > 2

```
1  {
2      "tasks": [
3          {
4              "type": "cppbuild",
5              "label": "C/C++: g++.exe build act
6              "command": "C:\\Users\\joey\\Downl
7              "args": [
8                  "-fdiagnostics-color=always",
9                  "-g",
10                 "${fileDirname}\\*.cpp",
11                 "-o",
12                 "${fileDirname}\\${fileBasenar
13             ],
14             "options": {
```

This setting means that your complier can **run all files in the current folder**.